

Dotscaper

사진을 도트 그래픽(픽셀아트) 스타일로 변환하는 순수 프론트엔드 웹 도구

강도 슬라이더 하나로 도트화 정도를 조절하고, 실시간으로 미리보며, PNG 로 내보냅니다.
모든 처리는 브라우저 안에서만 이뤄지며 사진은 서버로 전송되지 않습니다.

목차

1. 개요
 2. 기술 제약
 3. 핵심 변환 알고리즘
 4. 파라미터 매핑
 5. UI 구성
 6. 디자인 방향
 7. 엣지 케이스 & 품질 요구
 8. 개발 단계
 9. 완료 기준
 10. 향후 확장
 11. 부록: 참고 구현
-

1. 개요

사용자가 이미지를 올리면 강도 슬라이더 하나로 도트 그래픽 정도를 조절해 실시간 미리보기하고, PNG 로 내보내는 단일 페이지 도구입니다.

- **한 줄 정의:** “슬라이더 하나로 사진을 레트로 도트 그림으로.”
 - **처리 위치:** 100% 브라우저 내. 서버 전송 없음.
 - **부수 효과(중요):** Canvas 재출력 과정에서 원본의 EXIF/위치정보 메타데이터가 자동 제거됩니다. 프라이버시 이점으로 UI 에 명시합니다.
-

2. 기술 제약 (반드시 준수)

- 순수 HTML / CSS / JavaScript 만 사용합니다. 프레임워크·빌드 도구·npm 의존성이 없습니다.
 - 서버·백엔드가 없이 온디바이스 상태에서. 모든 연산은 클라이언트 Canvas 에서 처리하는 것을 목표로 설정합니다.
 - localStorage·sessionStorage 등 브라우저 스토리지를 사용하지 않습니다. 상태는 메모리(JS 변수)로만 관리합니다.
 - 최신 데스크톱/모바일 브라우저(Chrome, Safari, Firefox)에서 동작하며 반응형입니다.
-

3. 핵심 변환 알고리즘 (제품의 심장 — 정확히 구현)

아래 4 단계로 도트 그래픽을 생성합니다. 이 순서와 방식이 결과물 품질을 결정하므로 그대로 따릅니다.

1. **원본 로드** — 오프스크린 Canvas 에 그립니다.
2. **다운스케일 (해상도 축소)** — 긴 변이 gridLong px 가 되도록 종횡비를 유지하며 축소합니다. `imageSmoothingEnabled = true, imageSmoothingQuality = "high"` (평균 샘플링으로 부드럽게 축소).
3. **팔레트 축소 (색 개수 줄이기)** — 축소된 이미지를 colors 개 색으로 양자화합니다. **미디언 컷(median cut)** 알고리즘을 사용합니다(승인된 샘플이 이 방식으로 생성됨 — 반드시 median cut). **디더링을 사용하지 않습니다.** 각 픽셀을 팔레트 중 가장 가까운 색으로 매핑해 납작한 색 블록을 만듭니다. 디더링을 켜면 도트 느낌이 사라지므로 금지합니다.
4. **업스케일 (다시 확대)** — 원본(또는 표시) 크기로 확대합니다.
`imageSmoothingEnabled = false (nearest-neighbor) → 선명한 사각 도트.`

구현 메모: Canvas 에는 내장 색 양자화가 없으므로 median cut 을 직접 구현합니다. 픽셀 수는 다운스케일 후 작아서(수천 개 수준) 성능 문제가 없습니다.

4. 파라미터 매핑 (강도 → 격자/색)

슬라이더 강도 $s(0.0 \sim 1.0)$ 하나가 두 값을 동시에 제어합니다.

```
// 격자 해상도: 긴 변 기준 픽셀 수
const gridLong = Math.round(128 - (128 - 48) * s);
// 팔레트 색 개수
const colors = Math.round(32 - (32 - 8) * s);
```

강도	격자(긴 변)	색 개수	느낌
0%	128px	32 색	원본에 가까움 (약한 도트)
30%	104px	25 색	살짝 도트
60%	80px	18 색	밸런스 좋음 (권장 기본값)
100%	48px	8 색	최대 강도 (하한선)

하한선(100%) 근거: 55px/8 색은 거칠지만 여전히 그림으로 볼수있는 마지노선입니다. 이보다 낮추면(32px/4 색) 형체가 무너져 색덩어리가 됩니다. 여러 사진으로 검증해 확인한 값입니다.

구현 요구사항: 위 상수(128, 48, 32, 8)는 코드 상단에 설정 상수로 분리합니다. 실사용 후 미세조정(예: 하한을 40px/6 색)이 쉽도록 합니다.

기본 강도: 앱 로드 시 슬라이더는 60% 위치로 시작합니다.

알아둘 점(문서화만, 코드 변경 아님): 격자를 “긴 변” 기준으로 잡으므로 세로로 긴 사진은 짧은 변 방향이 더 거칠어집니다. 인물이 프레임에서 작으면 얼굴이 많이 뭉개집니다. 현재 스펙은 긴 변 기준으로 확정합니다. (짧은 변 기준은 향후 옵션으로 검토 — 10 장 참고.)

5. UI 구성

단일 화면, 큰 미리보기 + 간결한 컨트롤로 구성합니다.

컨트롤

- **사진 열기 버튼 + 드래그 앤 드롭**(캔버스 영역에 드롭). `accept="image/*"`.
- **강도 슬라이더**(range, 0~100). 값을 실시간 표시하며, 조작 시 미리보기를 즉시 갱신합니다(디바운스 `requestAnimationFrame`).
- **PNG 내보내기 버튼** → `canvas.toBlob()` → 다운로드 링크 자동 클릭. 파일명 예: `dotprint_<타임스탬프>.png`. 원본 해상도로 출력합니다(표시 크기 아님).

상태

- **초기 상태**: 업로드 전에도 빈 화면이 어색하지 않게 내장 샘플 이미지(절차적 생성 또는 간단한 도형 썸)를 띄워 효과를 바로 보여줍니다.
- **드래그 오버**: 드롭존 테두리를 강조합니다.
- **내보내기 중**: 버튼에 “렌더링 중...”을 표시합니다(큰 이미지 대비).

프라이버시 안내 문구

“모든 변환은 이 브라우저 안에서만 처리됩니다. 사진은 서버로 전송되지 않으며, 내보낸 이미지에서는 위치정보·촬영정보(EXIF)가 자동 제거됩니다.”

6. 디자인 방향

레트로 도트/8 비트 감성에 깔끔한 도구 UI 를 더합니다. (자매 도구 OVERPRINT 와 결이 통하되 별개 아이덴티티.)

- **팔레트**: 종이 톤 배경(`#e9e5d9` 계열) + 잉크 블랙 텍스트 + 강조색 1 개(예: 픽셀 그린 `#3ddc84`). 강조는 한 곳에만 씁니다.
- **타이포**: 제목은 굵은 디스플레이/픽셀 느낌, 라벨·수치는 모노스페이스. (CDN 금지이므로 `ui-monospace`, `monospace` 시스템 폰트로 처리.)
- **디테일**: 도트 그리드 모티프, 사각형(`border-radius` 최소), 픽셀 커서 등으로 주제(도트)를 반영합니다.
- **접근성**: 키보드 포커스 표시, 슬라이더 키보드 조작, `prefers-reduced-motion` 존중, 모바일까지 반응형.

폰트를 웹에서 불러오면 2 장의 “CDN 금지”와 충돌하므로, 시스템 폰트만으로 위 감성을 냅니다.

7. 엣지 케이스 & 품질 요구

- 매우 큰 이미지(예: 6000px+) 업로드 시에도 다운스케일이 먼저라 성능 문제가 없습니다. 단, 내보내기 원본 해상도에 상한(예: 긴 변 3000px)을 두어 과도한 메모리 사용을 방지합니다.
 - 세로/가로/정사각 모든 종횡비에서 격자·업스케일이 정확히 맞아야 합니다(경계에서 반 픽셀 잘림 없이).
 - 투명 PNG 입력 시 배경 처리를 정의합니다(투명 유지 또는 흰 배경 — 흰 배경 권장).
 - 이미지가 아닌 파일 드롭 시 무시합니다(에러 없이).
 - median cut 에서 색 수가 실제 고유색보다 많을 때를 안전 처리합니다.
-

8. 개발 단계 (마일스톤)

1. **1 단계 — 골격:** 레이아웃, 업로드(버튼+드래그), 캔버스에 원본 표시, 샘플 이미지.
 2. **2 단계 — 변환 엔진:** 다운스케일 → median cut 양자화(무디터) → nearest 업스케일. 강도 슬라이더 연결, 실시간 미리보기.
 3. **3 단계 — 내보내기:** 원본 해상도 렌더 → PNG 다운로드. EXIF 제거 확인.
 4. **4 단계 — 마감:** 디자인 적용, 프라이버시 안내, 반응형/접근성, 엣지 케이스.
-

9. 완료 기준 (Acceptance Criteria)

- ☐ 프레임워크·CDN·서버 없이 단일 index.html 로 동작합니다.
- ☐ 사진을 버튼/드래그로 올릴 수 있고, 강도 슬라이더로 실시간 도트 변환이 보입니다.
- ☐ 강도 0/30/60/100%에서 4 장 표의 격자/색 값이 정확히 적용됩니다.
- ☐ 100%에서도 결과가 “형체를 알아볼 수 있는” 도트 그림입니다(색덜어리로 무너지지 않음).

- □ 색 블록이 납작합니다(디더링 없음). median cut 팔레트입니다.
- □ PNG 를 원본 해상도로 내보낼 수 있고, 내보낸 파일에 EXIF 가 없습니다.
- □ 데스크톱·모바일 반응형이며 키보드 접근이 가능합니다.
- □ 코드 상단에 격자/색 하한 상수가 분리되어 조정 가능합니다.

10. 향후 확장 (이번 범위 아님 — 참고용)

- **강도 분리(고급 모드):** 격자와 색 개수를 각각 슬라이더로.
- **격자 기준 선택:** 긴 변 ↔ 짧은 변 (세로 인물 사진 대응).
- **팔레트 프리셋:** 자동 median cut 외에 고정 팔레트(게임보이 4 색, 리소 잉크 등).
- **부분 마스킹:** 드래그로 특정 영역만 강하게 처리(프라이버시 용도).
- **좌→우 스왑 애니메이션:** 원본→도트 전환을 WebM/GIF 로 내보내기.
- **OVERPRINT 연동:** 리소 하프톤 도구와 파이프라인 결합.

11. 부록: 참고 구현 (Python 프로토타입 — 로직 검증용)

아래는 승인된 결과물을 만든 파이썬 로직입니다. JS/Canvas 로 포팅합니다(파이썬을 실행하는 것이 아닙니다). median cut·무디더·nearest 업스케일의 기준으로 사용합니다.

```
from PIL import Image

def grid_for(s):    return round(128 - (128 - 48) * s)    # 격자(긴 변)
def colors_for(s):  return round(32 - (32 - 8) * s)        # 색 개수

def dotify(img, s):
    g, k = grid_for(s), colors_for(s)
    w, h = img.size
    scale = g / max(w, h)
    dw, dh = max(1, round(w*scale)), max(1, round(h*scale))
    small = img.resize((dw, dh), Image.BOX)                # 다운스케일
    (평균)
    small = small.quantize(colors=k, dither=Image.Dither.NONE) # median cu
    t, 무디더
    return small.convert("RGB").resize((w, h), Image.NEAREST) # nearest
업스케일
```